

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Dokumentace projektu do předmětu IFJ

Implementace překladače jazyka IFJ25

Tým: xkupcij00, Varianta: vv-BVS

Vedoucí: Jakub Jan Kupčík xkupcij00 25%

Petr Molík xmolikp00 25%

Jáchym Turek xturekj01 25%

Jan Kostečka xkostej00 25%

2. prosince 2025

Obsah

1	Rozdělení práce	2
2	Implementace	2
2.1	Lexikální analýza	2
2.2	Syntaktická analýza	2
2.3	Sémantická analýza	3
2.4	Generátor kódu	3
3	LL Gramatika	4
4	Konečný automat	6

1. Rozdělení práce

- Jakub Jan Kupčík - Sémantická analýza
- Petr Molík - Lexikální analýza
- Jáchym Turek - Generátor kódu
- Jan Kostečka - Syntaktická analýza

2. Implementace

2.1. Lexikální analýza

- `lex_scanner.c`, `lex_scanner.h`

Tato implementace lexikální analýzy vychází z mého vlastního řešení z minulého roku. Klíčové bylo nejprve navrhnout konečný automat (FSM), který definuje přechody mezi stavy na základě vstupních znaků. Samotná implementace je realizována funkcí `ifj_get_token`, která v cyklu načítá znaky a předává je přechodové funkci `transition`. Ta vyhodnocuje změnu stavu automatu.

U jednoduchých znaků (např. matematické operátory) dochází k okamžitému vytvoření tokenu a návratu do počátečního stavu. U složitějších struktur, jako jsou identifikátory nebo řetězce, se využívá dočasná dynamická paměť pro ukládání lexému. Pomocné funkce zajišťují detekci klíčových slov, vestavěných funkcí a převod “escape” sekvencí. Funkce `createToken` vytvoří token, který se skládá z jeho typu a lexému (řetězec načtených znaků před vytvořením tokenu). Do tokenu se tak uloží hodnota, kterou má daný lexém představovat a řetězec tak jak je zapsán na vstupu.

Speciální pozornost je věnována řetězcům, kde se pomocí čítače uvozovek rozlišuje mezi jednořádkovou a víceřádkovou variantou. Komentáře jsou analyzátořem ignorovány, přičemž u blokových komentářů je implementován čítač zanoření pro jejich správné uzavření. V případě lexikální chyby automat ukončí program s příslušným chybovým kódem.

2.2. Syntaktická analýza

- `parser.c`, `parser.h`, `ast.c`, `ast.h`

Syntaktický analyzátor jsem implementoval metodou rekurzivního sestupu, která vychází z mnou navržené LL gramatiky, doplněné o precedenční analýzu výrazů. Implementace je rozdělena do dvou klíčových částí: soubor `parser.c`, který zajišťuje samotnou analýzu a řízení překladu, a soubor `ast.c`, který obsahuje funkce pro konstrukci a správu uzlů abstraktního syntaktického stromu (AST).

Parser je organizován jako sada funkcí `parse_*`, z nichž každá odpovídá jednomu neterminálu v gramatice. Na vstupu pracuje se sekvencí tokenů získaných z lexikálního analyzátoru. Jednotlivé funkce podle aktuálního tokenu volají další části parseru nebo hlásí syntaktickou chybu (exit kód 2).

Struktura programu (povinný prolog, `class Program ...`) je zpracována ve funkcích `parse_prolog()` a `parse_class()`. Následuje analýza seznamu funkcí, kde parser rozlišuje tři typy deklarací: běžné funkce, gettery a settery.

Příkazy zpracovává funkce `parse_stmt()`. Ta rozhoduje, o jaký typ příkazu se jedná: deklarace proměnné, `if`, `while`, `return`, blok nebo příkaz začínající identifikátorem. U identifikátorů parser pomocí lookahead tokenu (nahlížení o jeden token dopředu) rozlišuje mezi přiřazením (`id = expr`) a voláním funkce (`id(...)`). Vestavěná funkce `Ifj.write` se zpracovává jako samostatná větev.

Výrazy používají precedenční syntaktický analyzátor implementovaný metodou binding power (`parse_expr_bp`). Ten zajišťuje správné vyhodnocení priorit operátorů a levé asociativity a slučuje operace do podoby AST uzlů.

Parser navíc přímo po vytvoření AST spouští sémantickou analýzu (funkce `semantic_firstpass` a `semantic_recurs`), která kontroluje definice, redefinice a typovou kompatibilitu.

Součástí syntaktické analýzy je průběžné vytváření abstraktního syntaktického stromu. Pro každý typ konstrukce jazyka existuje funkce `create_*`, která vrací nově alokovaný AST uzel. Modul `ast.c` zajišťuje:

- Bezpečnou alokaci paměti pro uzly a seznamy.
- Uložení metadat (typ uzlu, číslo řádku).
- Konstrukci konkrétních uzlů (např. blok, přiřazení, if-else, funkční volání, literály).
- Rekurzivní uvolnění celého stromu (`free_ast_node`), což je klíčové pro prevenci úniků paměti.

Parser kombinuje jednoduchost rekurzivního sestupu s robustností precedenční analýzy výrazů. Díky konstrukci AST během parsování je výsledek syntaktické analýzy plně strukturovaný a připravený pro generování cílového kódu. Implementace je modulární a snadno rozšiřitelná o nové jazykové konstrukce.

2.3. Sémantická analýza

- `parser.c`, `parser.h`, `symstack.c`, `symstack.h`, `symtable.c`, `symtable.h`, `err.c`, `err.h`

Sémantická analýza je implementována v souborech `'symtable'`, `'symstack'` a `'parser'`. Z důvodu opakování projektu a výběru stejného zadání (implementace tabulky symbolů pomocí výškově vyváženého binárního stromu) jsou kódy i komentáře v souborech `'symstack'` a `'symtable'` velmi podobné k mým odevzdaným souborům minulý rok.

Tabulka symbolů (`symtable`) je implementována jako výškově vyvážený binární strom. Symboly jsou ukládány pod jejich identifikátorem. Další data, která se o symbolu ukládají jsou:

1. Typ identifikátoru – proměnná nebo funkce
2. Datový typ symbolu (návratový typ u funkce)
3. Arita (u funkce)
4. Posun v paměti (pro generátor kódu)

Pro analýzu vnořenosti a rozsahů platnosti je implementován zásobník (`symstack`), na který jsou vloženy tabulky symbolů. Nejprve globální tabulka, kde jsou uloženy deklarace funkcí. Následující tabulky jsou určeny pro lokální proměnné.

Sémantická analýza využívá syntaktickou analýzou vygenerovaný AST strom. Nejprve projde všechny deklarace funkcí a zkontroluje, zda nejsou některé z nich reдекlarovány. Funkce s jejich aritou jsou uloženy do globální tabulky symbolů. Toto slouží k tomu, že funkce může být volána i před její definicí. Dále sémantická analýza probíhá v rekurzivní funkci. Při zjištění sémantické chyby vrací správný návratový kód dle typu chyby.

2.4. Generátor kódu

- `codegen.c`, `codegen.h`

Generátor kódu je implementovaný v `codegen.c`. Vstupem je kořen AST stromu, který se prochází celkově dvakrát. Poprvé se strom prochází ve funkci `first_travers` pro nalezení globálních proměnných, které se v zdrojovém kódu nemusí na začátku deklarovat, ale v mezikódu ano do globálního rámce GF. Druhý průchod už rekurzivně prochází každý uzel a generuje mezikód IFJcode25. Na začátku programu se vytisknou pomocné globální proměnné a předdělané funkce jako `IFJ_WRITE`. Poté se tisknout globální proměnné ze zdrojového kódu, které jsou uloženy ve stringovém poli. Následně se

vyhodnocují jednotlivé uzly a to primárně přes zásobníkové pokyny v IFJcode25. Jelikož je ve zdrojovém kódu pro čísla jenom typ Num, je všechno implementováno ve floatech. Jelikož je zdrojový kód dynamicky typovaný, musí se kontrolovat typy při operacích, k tomu jsou primárně využívány pomocné globální proměnné. Pro případný shadowing je využíván offset, který se dává za každou proměnnou a pro návěští je využíván `if_label_counter`, který se dává za každé nové návěští u `if`, `while` a tam kde se kontrolují typy.

3. LL Gramatika

1. `<prog>` $\rightarrow$ `import "ifj25" for Ifj class Program { <func_list> }`
2. `<func_list>` $\rightarrow$ `static id <func_rest> <func_list>`
3. `<func_list>` $\rightarrow$ ε
4. `<func_rest>` $\rightarrow$ `{ <block_content> }`
5. `<func_rest>` $\rightarrow$ `( <params> ) { <block_content> }`
6. `<func_rest>` $\rightarrow$ `= ( id ) { <block_content> }`
7. `<params>` $\rightarrow$ `id <params_n>`
8. `<params>` $\rightarrow$ ε
9. `<params_n>` $\rightarrow$ `, id <params_n>`
10. `<params_n>` $\rightarrow$ ε
11. `<block>` $\rightarrow$ `{ <block_content> }`
12. `<block_content>` $\rightarrow$ `<stmt_list> }`
13. `<stmt_list>` $\rightarrow$ `<stmt> <stmt_list>`
14. `<stmt_list>` $\rightarrow$ ε
15. `<stmt>` $\rightarrow$ `var id`
16. `<stmt>` $\rightarrow$ `if ( <expr> ) <block> <else_part>`
17. `<stmt>` $\rightarrow$ `while ( <expr> ) <block>`
18. `<stmt>` $\rightarrow$ `return <expr_opt>`
19. `<stmt>` $\rightarrow$ `<block>`
20. `<stmt>` $\rightarrow$ `id <stmt_id_tail>`
21. `<stmt>` $\rightarrow$ `Ifj.write ( <args> )`
22. `<else_part>` $\rightarrow$ `else <block>`
23. `<else_part>` $\rightarrow$ ε
24. `<stmt_id_tail>` $\rightarrow$ `= <expr>`
25. `<stmt_id_tail>` $\rightarrow$ `( <args> )`
26. `<args>` $\rightarrow$ `<expr> <args_n>`
27. `<args>` $\rightarrow$ ε
28. `<args_n>` $\rightarrow$ `, <expr> <args_n>`
29. `<args_n>` $\rightarrow$ ε
30. `<expr_opt>` $\rightarrow$ `<expr>`
31. `<expr_opt>` $\rightarrow$ ε

	import	static	}	var	if	while	return	{	id	Ifj.write	else	\$ (EOF)
<code><prog></code>	1											
<code><func_list></code>		2	3									
<code><stmt_list></code>			14	13	13	13	13	13	13	13		
<code><stmt></code>				15	16	17	18	19	20	21		
<code><else_part></code>			23	23	23	23	23	23	23	23	22	23

Tabulka 1: LL(1) tabulka pro neterminály `<prog>`, `<func_list>`, `<stmt_list>`, `<stmt>` a `<else_part>`. Čísla v buňkách odkazují na čísla produkcí gramatiky.

	* /	+-	rel	is	==	!=	ID	(	)	\$
* /	>	>	>	>	>	<	<	>	>	
+-	<	>	>	>	>	<	<	>	>	
rel	<	<	>	>	>	<	<	>	>	
is	<	<	<	>	>	<	<	>	>	
== !=	<	<	<	<	>	<	<	>	>	
ID	>	>	>	>	>			>	>	
(	<	<	<	<	<	<	<	=		
)	>	>	>	>	>			>	>	
\$	<	<	<	<	<	<	<		=	

Tabulka 2: Precedenční tabulka pro syntaktickou analýzu výrazů. Symboly < a > značí vztah precedence (shift/reduce), E nedovolenou kombinaci a = párování (např. mezi (a)).

4. Konečný automat

Obrázek 1: Konečný automat

